Assignment #1 — Graph Traversals

Artificial Intelligence and Knowledge Engineering
SI4024L, 2025/2026

Oscar Lopez

April 13, 2026

Contents

1 Introduction **2**

2 Data and Graph **2**
2.1 GTFS Files . . . 2
2.2 Filtering Services by Date . . . 2
2.3 Graph Structure . . . 2
2.4 Transfer Penalty . . . 3

3 Task 1: Shortest Path **3**
3.1 Search State . . . 3
3.2 Dijkstra . . . 3
3.3 A* . . . 3
3.4 Bidirectional A* . . . 4
3.5 Algorithm Comparison . . . 4

4 Task 2: TSP with Tabu Search **5**
4.1 Problem . . . 5
4.2 Initial Solution . . . 5
4.3 Tabu Search (2a — Basic) . . . 5
4.4 Variable Tabu List Size (2b) . . . 5
4.5 Aspiration Criterion (2c) . . . 6
4.6 Neighbourhood Sampling (2d) . . . 6
4.7 Variant Comparison . . . 6

5 Conclusions **6**

1 Introduction

This assignment implements pathfinding algorithms on a real railway timetable: the Koleje Dolnośląskie network in Poland, provided in GTFS format.

Two tasks are solved:

- **Task 1:** Find the best route between two stops, minimising either travel time (`t`) or number of transfers (`p`), using Dijkstra and A*.
- **Task 2:** Given a start stop and a list of stops to visit, find the shortest closed tour using Tabu Search.

The main challenge is that the graph is **time-dependent**: the next available train depends on when you arrive at a stop, so the cost of a route changes throughout the day.

For this report, I used artificial intelligence as a support tool to better understand the assigned tasks and to improve the overall structure of my code. It helped me clarify concepts, organize my approach more efficiently, and ensure that my solutions were logically structured and consistent. Additionally, AI assisted me in generating clear and coherent documentation, as well as providing guidance during the writing process of this report, making the final result more comprehensive and well-presented.

2 Data and Graph

2.1 GTFS Files

The GTFS dataset consists of CSV files describing the timetable:

File	Content
`stops.txt`	Stop names, GPS coordinates, platform grouping
`routes.txt`	Train lines (D1, D6, D66, ...)
`trips.txt`	Individual runs of each line
`stop_times.txt`	Departure and arrival times per stop per trip
`calendar.txt`	Which days each service runs
`calendar_dates.txt`	Exceptions (holidays, extra services)

Table 1: GTFS files used in this assignment.

2.2 Filtering Services by Date

Before building the graph, only the trips active on the query date are loaded. A service is active if its weekday flag in `calendar.txt` matches the query day, and no exception in `calendar_dates.txt` cancels it. This ensures that Wednesday and Saturday return different routes.

2.3 Graph Structure

The graph nodes are stop platforms. Each edge represents one segment of a trip:

$$\text{edge} = \langle \text{from_stop},\ \text{to_stop},\ t_{\text{dep}},\ t_{\text{arr}},\ \text{trip_id},\ \text{route} \rangle$$

Edges are stored sorted by departure time, so the next available departure from any given time can be found in $O(\log n)$ with binary search.

2.4 Transfer Penalty

Changing trains at a station requires walking to another platform. A minimum transfer time of 2 minutes is enforced: a departure on a different train is only considered if it leaves at least 120 seconds after the passenger's arrival.

3 Task 1: Shortest Path

3.1 Search State

The search state is the pair $(\text{stop}, \text{trip})$. Tracking the current trip is necessary to correctly apply the transfer penalty: staying on the same train has no penalty, while boarding a different one requires at least 2 minutes.

3.2 Dijkstra

Dijkstra explores states in order of increasing cost, always expanding the cheapest known state next.

Criterion `t` — travel time. The cost of a state is the arrival time in seconds. The algorithm finds the state at the destination with the earliest arrival, and the travel time is:

$$\text{cost} = t_{\text{arrival}} - t_{\text{start}}$$

Criterion `p` — transfers. The cost is the pair (transfers, t_{arrival}), compared lexicographically: fewer transfers always wins, and arrival time breaks ties. A transfer is counted each time the trip changes at a stop.

3.3 A*

A* improves on Dijkstra by using a heuristic $h(n)$ that estimates the remaining cost to the goal. Each state is prioritised by:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost accumulated so far. This guides the search towards the destination and reduces the number of states explored.

Admissibility. For A* to guarantee an optimal solution, the heuristic must never *overestimate* the true remaining cost. This property is called **admissibility**.

Heuristic for criterion `t`.

$$h(n) = \frac{d_{\text{straight line}}(n,\ \text{goal})}{V_{\max}}$$

The straight-line distance between two GPS coordinates (haversine formula) divided by $V_{\max} = 50$ m/s (≈ 180 km/h) gives a lower bound on the remaining travel time: no train travels faster than $V_{\max}$, so the actual time is always at least this value. The heuristic is therefore admissible.

Heuristic for criterion `p`.

$$h(n) = 0$$

The zero heuristic is always admissible (transfers remaining ≥ 0). With this heuristic A* behaves identically to Dijkstra for the transfer criterion, since no additional guidance is available without pre-computing a transfer distance matrix.

3.4 Bidirectional A*

Bidirectional A* runs two searches simultaneously: one forward from the origin and one backward from the goal. They stop when they meet in the middle.

The main challenge is that the backward search cannot use real train schedules (trains do not run backwards). Instead, it uses the sum of edge durations to estimate how far back it has reached. When the two frontiers meet, a final forward A* is run from the meeting point to the goal to produce the actual feasible path.

This approach explores roughly half the nodes of a unidirectional search on long-distance queries.

3.5 Algorithm Comparison

The following command was used to generate the comparison data:

```
GTFS_DATE=2026-03-04 python3 solution.py \
    "Wrocław Główny" "Jelenia Góra" t 06:00:00 --compare
```

Algorithm	Cost	Nodes visited	Time (ms)
Dijkstra	146.0 min	684	17.5
A*	146.0 min	499	16.6
Bidirectional A*	146.0 min	159	11.3

Table 2: Algorithm comparison on query *Wrocław Główny* → *Jelenia Góra*, criterion `t`, departure 06:00, Wednesday 2026-03-04.

All algorithms find the same optimal cost. A* visits fewer nodes than Dijkstra because the heuristic prunes states that are far from the destination. Bidirectional A* reduces the search space further.

4 Task 2: TSP with Tabu Search

4.1 Problem

Given a starting stop A and stops $L = \{p_1, \ldots, p_n\}$ to visit, find the ordering of L that minimises the total cost of the round trip $A \to \cdots \to A$. There are $n!$ possible orderings, so an exhaustive search is only feasible for very small n.

The cost of the tour is computed by chaining A* calls: each segment departs at the actual arrival time of the previous one. This means the cost is **time-dependent** and **asymmetric**: visiting Legnica before Jelenia Góra may cost more or less than the reverse order, because the available trains differ.

4.2 Initial Solution

A greedy nearest-neighbour heuristic builds the starting tour: from the current stop, always go next to the closest unvisited stop (by A* cost). This gives a reasonable starting point for Tabu Search.

4.3 Tabu Search (2a — Basic)

Tabu Search is a local search that avoids revisiting recent solutions by keeping a **tabu list** of recently used moves.

Algorithm 1 Tabu Search

```
s ← initial solution,   s* ← s
T ← empty tabu list
for each iteration do
    Generate all 2-opt neighbours of s
    s' ← best neighbour whose move is not in T
    Add move of s' to T
    s ← s'
    if cost(s') < cost(s*) then s* ← s'
    end if
end for
return s*
```

The **2-opt** neighbourhood works by reversing a sub-sequence of the tour. For n stops there are $\binom{n}{2}$ possible 2-opt moves.

In variant 2a the tabu list grows unboundedly, so no move is ever repeated.

4.4 Variable Tabu List Size (2b)

A fixed-size FIFO list of size T is used instead. When the list is full, the oldest move is removed. The following sizes are compared:

T	**Effect**
∞ (unbounded)	No move is ever repeated
$\sqrt{n}$	Small list, more freedom to explore
n	Balanced
$2n$	Large list, less risk of cycling

Table 3: Tabu list sizes compared in variant 2b.

4.5 Aspiration Criterion (2c)

The aspiration criterion allows a tabu move if it produces a new global best:

$$\text{allow tabu move} \iff \text{cost(candidate)} < \text{cost}(s^*)$$

The reasoning is that if a tabu move leads to a better solution than any seen before, it cannot be creating a cycle. Blocking it would be overly restrictive.

4.6 Neighbourhood Sampling (2d)

Instead of evaluating all $\binom{n}{2}$ neighbours, only a random fraction ρ is evaluated per iteration. With $\rho = 0.5$, each iteration is twice as fast, allowing more iterations in the same time at the cost of missing some neighbours in each step.

4.7 Variant Comparison

The following command was used to generate the comparison data:

```
GTFS_DATE=2026-03-04 python3 solution.py \
    "Wrocław Główny" "Jelenia Góra;Legnica;Wałbrzych Główny" \
    t 06:00:00 --task2 --tabu-compare
```

Configuration	**Cost (min)**	**Iters**	**Asp.**	**Time (ms)**
Unbounded T	345.0	4	0	330
$T = \sqrt{n} = 2$	345.0	200	0	12596
$T = n = 4$	345.0	4	0	326
$T = 2n = 8$	345.0	4	0	321
+ Aspiration	345.0	4	0	327
+ 50% sample	424.0	3	0	144

Table 4: Tabu Search variant comparison on *Wrocław Główny* → *{Jelenia Góra, Legnica, Wałbrzych Główny}*, criterion `t`, departure 06:00, Wednesday 2026-03-04.

5 Conclusions

- Tracking `trip_id` in the search state is essential to handle transfers correctly, including the minimum transfer penalty.

- The haversine heuristic for A* is admissible and reduces the number of nodes explored compared to Dijkstra, while guaranteeing the same optimal solution.
- Computing tour costs with time-dependent A* (instead of a static distance matrix) is necessary for correct asymmetric TSP costs.
- Among the Tabu Search variants, the aspiration criterion has the greatest practical impact by recovering solutions only reachable through tabu moves.